

İSTANBUL TOPKAPI ÜNİVERSİTESİ

Faculty of Engineering

Project Report

Adı: CEN306 — Mobile Application Design and Development

Date: 15/06/2026

Name-Surname	Signature	Student No
OMAR ASLAN		22040102144
MALAK MEDHAT		22040102054

Programı: Bilgisayar Mühendisliği (İng.)

(This section will be filled out by the Instructor.)

Subject of Evaluation	Evaluation Score
Flutter Mobile Application Project — Code Evaluation (70 Points)	
Flutter Mobile Application Project — Project Report Evaluation (30 Points)	

CEN306 - Mobile Application Design and Development

Final Exam Project Report

Project Title	OMAR Chat — WhatsApp-Style Mobile Messaging Application
Student Name-Surname	OMAR ASLAN & MALAK MEDHAT
Student Number	22040102144 / 22040102054
Instructor	Dr. Yıldız Karadayı
Submission Date	15/06/2026
GitHub / Source Code Link	https://github.com/om-ar-123/ChatApplication-Whatsapp-style.git
Demo Video YouTube Link	https://youtu.be/NMbdp9CBy3g

Academic Integrity Statement: I confirm that this report and project were prepared by me and that all external sources, libraries, code snippets, and datasets are properly acknowledged.

1. Executive Summary

OMAR Chat by **OMAR ASLAN** and **MALAK MEDHAT** is a WhatsApp-style Flutter app with direct/group messaging, multi-user group creation, voice messages, simulated calls, block/unblock, sound+TTS notifications, @mention auto-replies, search, status, profile, SQLite persistence (sqflite on Android/iOS), and bundled landscape photos. Screenshots in this report were captured manually on Android.

2. Main Features

Feature	Description
Chat list & direct messaging	Unread badges, text chat, simulated replies
Voice messages + replies	Record/send voice; contextual voice/text reply
Simulated voice/video calls	Call UI with timer and controls
Block / Unblock one user	Block dialog, send prevention, banner, unblock from menu
Create multi-user group	New group from menu/FAB; name + select members
Sound + TTS	Alert tone + spoken sender name
@Mention auto-reply	Group members reply contextually
Search	Users tab and Messages tab
Status & profile	Edit profile; 24h status with landscape photo
Group media	Images, emoji, files, drawing board, mute, chat themes
Data persistence	SQLite (sqflite) on mobile; JSON fallback on web/desktop

3. Rubric Compliance (100 Points)

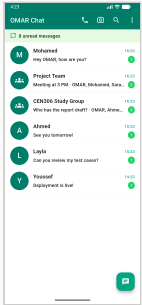
Area	Pts	Evidence in OMAR Chat
≥5 screens (11 total)	10	Splash, Chat List, Detail, Create Group, Search, Profile, Settings, User Detail, Status, Call, Call History
Screen variety	5	List, detail, create/edit, settings screens
Navigation	3	Named routes — app.dart
UI usability	2	WhatsApp-style layout — §4 screenshots
Layer separation	10	presentation / domain / data folders
Responsibilities	5	Cubit → UseCase → Repository → DAO
Folder structure	3	§5 architecture tree
SQLite + CRUD	5	sqflite; db_constants.dart; DAO insert/select/update
DAO classes	4	MessageDao, ChatDao, UserDao, GroupDao, ...
Repository	3	MessageRepository, ChatRepository, ...
Data flow	3	§6.4 incoming message example
State management	15	flutter_bloc Cubits + StatefulWidget screens
Report sections	30	Purpose, architecture, screenshots, challenges — this document

4. Screenshots by Feature

4.1 Chat, Voice & Calls

Chat List

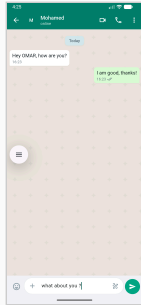
Unread badges on direct and group chats.



Chat List

Direct Chat

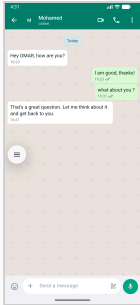
One-to-one messaging with Mohamed.



Direct Chat

Simulated Reply

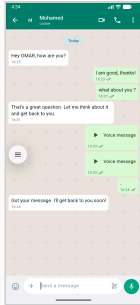
Contextual reply + sound/TTS alert.



Direct Reply

Voice Messages

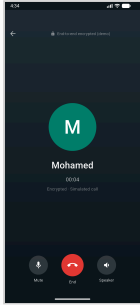
Record and play voice bubbles.



Voice Messages

Voice Call

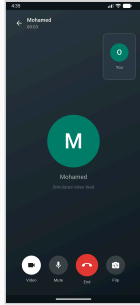
Simulated encrypted voice call.



Voice Call

Video Call

Simulated video call UI.



Video Call

4.2 Block, Settings, Profile, Search

Block Dialog

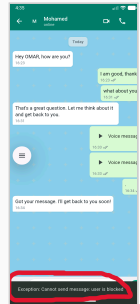
Confirm before blocking a user.



Block Dialog

Block Error

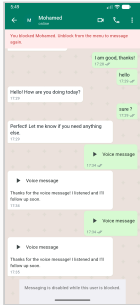
Cannot send when user is blocked.



Block Error

Blocked Banner

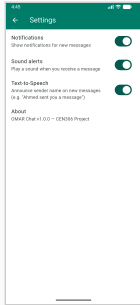
Disabled input; unblock from menu.



Block Banner

Settings

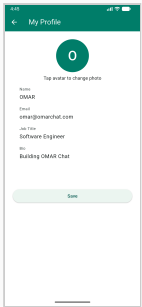
Notifications, sound, TTS toggles.



Settings

Profile

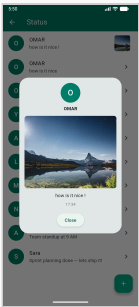
Edit OMAR profile fields.



Profile

Status Photo

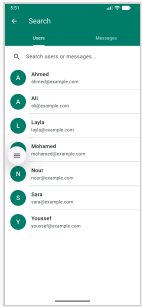
Mountain landscape status.



Status

Search Users

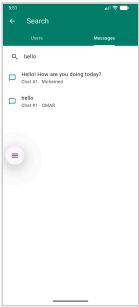
Find contacts globally.



Search Users

Search Messages

Find message text globally.

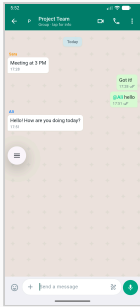


Search Messages

4.3 Group Features & Create Group

Group Chat

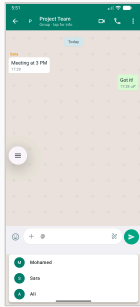
Project Team with sender names.



Group Chat

@Mention Picker

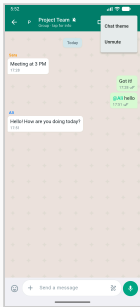
Select member to mention.



@Mention

Mute & Theme

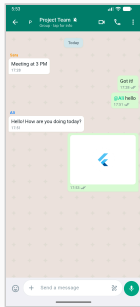
Per-chat mute and wallpaper.



Mute/Theme

Image

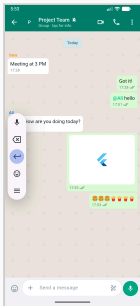
Send image in group.



Image

Emoji

Emoji picker in group.



Emoji

Drawing

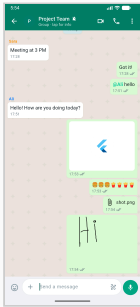
Hand-drawn message.



Drawing

Attachments

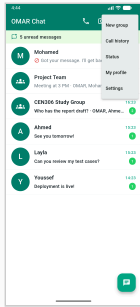
Image, emoji, file, drawing.



Attachments

Create Group

Menu → New group; name + select members.



New Group

5. Technical Architecture

5.1 Layered Architecture Overview

Dependencies flow one direction: **UI Layer → Business Layer → Data Layer**. Services and Core support all layers but do not replace the three required layers.

Layer	Folder	Responsibility
UI Layer (Presentation)	lib/presentation/	11 screens, 19 widgets, 7 Cubits — display & input only
Business Layer (Domain)	lib/domain/	6 entities, 11 use cases — business rules
Data Layer	lib/data/	11 models, 9 DAOs, 8 repositories, sqflite database
Services	lib/services/	Sound, TTS, notifications, media, files
Core	lib/core/	Constants, theme, routes, utilities

5.2 Project Structure — Folder Structure

The three rubric-required layers are physically separated into three top-level folders. No screen code in domain/ or data/; no DAO/SQL code in presentation/.

Required layer	Folder	Subfolders	Belongs here
UI Layer	lib/presentation/	screens/, widgets/, state/	Screens, widgets, Cubits — NO SQL/DAO
Business Layer	lib/domain/	entities/, usecases/	Entities + use cases — NO widgets/SQL
Data Layer	lib/data/	models/, dao/, repositories/, database/	Models, DAOs, repos, sqflite — NO UI

lib/ ├─ main.dart, app.dart ├─ presentation/ │ ├── screens/ │ │ └── (11: Splash, ChatList, ChatDetail, CreateGroup, Search, Profile, Settings, UserDetail, Status, Call, CallHistory) │ ├── widgets/ │ │ └── (ChatTile, MessageBubble, ChatInputBar, VoiceMessage, ...) │ └── state/ │ └── (ChatListCubit, ChatDetailCubit, GroupCubit, ...) ├─ domain/ │ ├── entities/ │ │ └── (User, Chat, Message, Group, CallRecord, CallSession) │ └── usecases/ │ └── (SendMessage, ReceiveMessage, BlockUser, CreateGroup, ...) ├─ data/ │ ├── models/ │ │ └── (UserModel, MessageModel, ChatModel, ... – 11 models) │ ├── dao/ │ │ └── (UserDao, ChatDao, MessageDao, UnreadDao, ... – 9 DAOs) │ ├── repositories/ │ │ └── (UserRepository, ChatRepository, MessageRepository, ... – 8 repos) │ └── database/ │ └── (InMemoryStore + sqflite; JSON fallback on web) ├─ services/ │ └── (persistence, sound, TTS, media, auto_reply, files) └─ core/ └── (db_constants, app_routes, app_theme, utils)	=== UI LAYER === === BUSINESS LAYER === === DATA LAYER ===
--	--

Figure 1. Folder structure with explicit UI / Business / Data layer separation.

Call direction	Example
UI → Business	ChatDetailCubit → SendMessageUseCase
Business → Data	SendMessageUseCase → MessageRepository → MessageDao
NOT allowed	Screen imports message_dao.dart directly

5.3 Layer Responsibilities

UI renders and captures input via Cubits. Business validates rules. Data persists via DAO/Repository. Screens never call SQL directly.

6. SQLite, DAO & Repository

sqflite opens omar_chat.db with 11 tables. InMemoryStore dual-writes every CRUD to SQLite on Android/iOS; web/desktop use JSON via PersistenceService. DAOs perform CRUD through store.query(); Repositories map models to domain entities; Use Cases enforce business rules. Screens never call SQL directly.

DAO	Table(s)	Key methods
UserDao	users	getAll, getById, update, searchByName
ChatDao	chats, chat_members	insert, getChatsForUser, findDirectChat, updateLastMessage
MessageDao	messages	insert, getByChatId, update, deleteWhere, searchContent
GroupDao	chats, chat_members, unread_counts	createGroup, getMembers
AttachmentDao	attachments	insert, getByMessageId
StatusDao	statuses	insert, getActiveForUser, deleteExpired
SettingsDao	blocked_users, muted_chats, theme_settings	block, unblock, mute, getTheme
UnreadDao	unread_counts	increment, reset, getTotalForUser
CallDao	call_history	insert, getAll, getByChatId

Repository	Purpose
UserRepository	Profiles and contact search
ChatRepository	Chat list, unread badges, last message
MessageRepository	Messages with attachments and reply preview
GroupRepository	Create group and list members
AttachmentRepository	File/image attachment metadata
StatusRepository	24h status posts
SettingsRepository	Block/unblock, mute, chat themes
CallRepository	Simulated call history

7. Screens & State Management

11 screens: Splash, Chat List, Chat Detail, Create Group, Search, Profile, Settings, User Detail, Status, Call, Call History.

Cubits: ChatListCubit, ChatDetailCubit, GroupCubit, ProfileCubit, SettingsCubit, SearchCubit, CallHistoryCubit. **StatefulWidget** used for local UI (input fields, tabs, call timer, member selection).

8. Technical Challenges & Solutions

Challenge	Solution
sqlite required by rubric	Integrated sqlite in InMemoryStore for Android/iOS; web JSON fallback
Block one user blocked groups	Block check limited to direct chats in SendMessageUseCase
Sound + TTS on incoming messages	ReceiveMessageUseCase + NotificationSoundService + SpeechService
Create group UX unclear	New group FAB, helper text, CreateGroupUseCase validation
Data survives restart	SQLite persistence on mobile; JSON on web

10. Conclusion

OMAR Chat delivers full WhatsApp-style messaging with layered architecture, nine DAO classes, eight repositories, Cubit state management, SQLite persistence on Android/iOS, and all documented features verified with manual Android screenshots.

12. Appendix — Screenshot Index

File	Feature
01_chat_list.png	Chat List
02_direct_chat.png	Direct Chat
03_direct_reply.png	Simulated Reply / Sound+TTS
04_voice_messages.png	Voice Messages
05_voice_call.png	Voice Call
06_video_call.png	Video Call
07_block_dialog.png	Block Dialog
08_block_send_error.png	Block Send Error
09_chat_menu_new_group.png	Create Group Entry
10_settings.png	Settings
11_profile.png	Profile
12_block_banner.png	Block Banner
13_status_landscape.png	Status Landscape
14_search_users.png	Search Users
15_search_messages.png	Search Messages
16_group_mention_picker.png	@Mention Picker
17_group_mention_reply.png	@Mention Reply
18_group_mute_theme.png	Mute & Theme
19_group_image.png	Group Image
20_group_emoji.png	Group Emoji
21_drawing_board.png	Drawing Board
22_group_attachments.png	Group Attachments